

Computer Programming: Lecture 7

Outline

- Strings

Strings

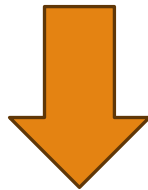
- A string in Python is a sequence of characters. It can be enclosed by either single or double quotes, although single quotes are more commonly used. For example, the following are all strings:

```
'This is a string'  
"This is also a string"  
"""Multiline  
string  
"""
```

Strings

Strings can contain any character, including letters, numbers, punctuation, and whitespace. They can also contain special characters, such as escape sequences. For example, the following string contains an escape sequence that represents a newline character:

```
print('This is a string\nwith a newline character')
```



```
This is a string  
with a newline character
```

String Methods

- `len()`: Returns the length of the string.
 - `name = "hello "`
`print(len(name)) -> 6`
- `+`: can be used to concatenate strings
`string = "Hello " + "world"`
`print(string) => Hello world`
- `*`: can be used to repeat strings
`string = "hello "`
`print(string * 2) -> hello hello`

String Methods

- `upper()`: Converts the string to uppercase.

```
name = "Abebe"
```

```
print(name.upper()) => ABEBE
```

- `lower()`: Converts the string to lowercase.

```
name = "Abebe"
```

```
print(name.lower()) => abebe
```

`capitalize()`: it changes the first letter of the string to uppercase.

```
my_string = "hello world"
```

```
print(my_string.capitalize()) -> Hello world
```

`title()`: it changes the starting letter of each word in a string to uppercase.

```
my_string = "hello world"
```

```
print(my_string.title()) -> Hello World
```

String Methods

- `startswith()`: Returns True if the string starts with a specified substring
name = "Abel"
print(name.startswith("ab")) => False
- `endswith()`: Returns True if the string ends with a specified substring
name = "Abel"
print(name.endswith("bel")) => True
- `in`: membership testing
fruit = "banana"
print("ana" in fruit) -> True

String Methods

- `find()` -> returns the index of the first occurrence of a substring and doesn't raise an error if the substring is not found.

```
fruit = "banana"
```

```
print(fruit.find("ana")) ->1
```

- `index()` -> returns the index of the first occurrence of a substring but raises a value error if the substring is not found.

```
fruit = "banana"
```

```
print(fruit.index("ana")) -> 1
```

- `replace()`: Replaces all occurrences of a substring with another substring.

```
mystring = "Hello world Hello"
```

```
replaced = mystring.replace("Hello","Hi")
```

```
print(replaced) => Hi world Hi
```

String Methods

Strip(): Clears leading and trailing whitespaces.

```
my_string = " hello "  
print(my_string.strip()) -> hello
```

lstrip(): clears leading whitespaces.

```
my_string = " hello"  
print(my_string.lstrip()) -> hello
```

rstrip(): clears trailing whitespaces.

```
my_string = "hello "  
print(my_string.rstrip()) -> hello
```

String Operations

- `split()`: Splits the string into a list of substrings based on a delimiter.

```
mystring = "Hello world"
```

```
mylist = mystring.split(" ")#split by space
```

```
print(mylist) =>['Hello', 'world']
```

- `join()`: Joins a list of strings together with a delimiter.

```
my_list = ["hello","world","hi"]
```

```
new_string = "#".join(my_list)#join by hyphen
```

```
print(new_string)=>hello#world#hi
```

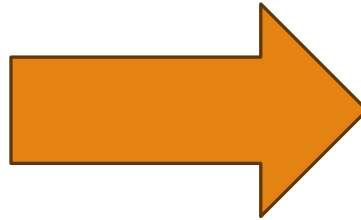
Indexing a string

- To index a character we use square brackets [] and use an integer starting from 0 to the length of the string minus one.
 - The first character is indexed by 0.
 - The last character is indexed by length of the string minus 1.

```
name = "Abel"  
len(name) -> 4  
print(name[0]) => A  
print(name[1]) => b  
print(name[2]) => e  
print(name[3]) => l
```

Traversing a string with a while loop

```
i = 0
name = "Abebe"
while(i < len(name)):
    print(name[i])
    i = i + 1
```



A
b
e
b
e

```
var = "python"
for i in var:
    if(i == "h"):
        continue
    print(i)
```

Traversing a string using a for loop

```
name = "Abebe"  
for letter in name:  
    print(letter)
```



A
b
e
b
e

Slicing

- A substring of a string is obtained by taking a slice.
- We use the colon operator to get a continuous section of the string

Syntax

[start index:end index]

Outputs

A substring from the start index to
the end index minus 1

Example

M	o	n	t	y		P	y	t	h	o	n
0	1	2	3	4	5	6	7	8	9	10	11

```
s = 'Monty Python'
```

```
print(s[0:4]) => Mont
```

Example

`print (s[8:]) =>thon`

#if a value is not provided the default start index is 0 and the end index is length - 1

`print (s[:]) => Monty Python`

`print(s[:4]) => Mont`